

In-House Device Testing

Atomica Corporation

Introduction

At Atomica test requests are made by customers of devices (INEZ, Island, Nautilus, Midnight...) and there is a lost revenue stream when we don't provide this service. Atomica has the key equipment necessary to perform both in-line and end-of-line testing, but it does not possess the expertise. This document lays out a pathway to develop a system that can be integrated into FabOps that will enable technicians of modest technical expertise to set up and perform the most common test types requested by the customer.

Contents

Introduction	1
Objectives	3
Challenges	3
Mitigation	4
Plan	5
ATA (Atomica Test Application)	5
Vision System	5
Implementation	7
Software stack.....	8
Project structure and implementation	8
Appendix (i): Wafer-Test Planning and Automation System	11
1. Inputs the ATA software will need	11
Wafer map	11
Device electrical model	11
Pad layout.....	12
Alignment mark data.....	12
Bench equipment definition	12
Prober definition.....	12

2. Main software modules.....	13
A. Wafer map parser.....	13
B. Coordinate transform engine.....	14
C. Vision/alignment module	15
D. Test-plan generator	16
E. Instrument-control layer.....	18
F. Prober driver abstraction.....	19
3. Important safety/state logic.....	20
Appendix ii.....	21
Practical development order	21
Phase 1: Offline test-plan generator	21
Phase 2: Instrument-only control	21
Phase 3: Prober dry-run mode.....	21
Phase 4: Single-die probing.....	22
Phase 5: Alignment correction	22
Phase 6: Full-wafer automated probing.....	22
The key data model	22
Example generated test flow	23
Best overall design	23
Appendix iii Full code simulation	25
1. Python: the main application/control language	25
2. PyVISA for GPIB control.....	27
3. NumPy for coordinate transforms	30
4. Pandas for wafer maps and results	33
5. OpenCV for alignment mark recognition.....	35
6. SQLite or PostgreSQL for test data	38

Objectives

The objective is to set up a clear and intuitive path that technicians, engineers and TLs can follow to allow in-line and end-of-line wafer level electronics device testing of both pure tessellated and 'pizza' wafers. Electronic parameters that can be measured will be resistance, leakage current, capacitance, inductance and impedance. The option to measure RF and semiconductor (e.g., diode and transistor) parameters will not be considered at this stage, as to date there has been no requirement, however it is possible and an attainable goal in the future. No allowance is being made for temperature measurement or device heating/cooling as the equipment that Atomica has available cannot offer this service. It should be noted though that this is a service that is requested and could also be accounted for in the future.

Challenges

The challenges with this project are;

1. Programming the prober (8) Atomica uses for wafer probing (the Accretech UF200R) with the wafer layout (this has historically been by adapting LabVIEW projects)
2. Operating the prober (Ideally this will be completely automated)
3. Operating the measurement instruments; Keithley 707B – Switching Matrix, Keysight 34461A - Digital Multimeter, Keithley 2636B - System SourceMeter, Keysight 33500B - Waveform Generator. (All instruments will also ideally be automated)
4. Setting up the test electronics (this will require some electronics understanding)
5. Operating the test electronics (ideally automated)
6. Setting up generic algorithms and control software to run the complete system
7. Interfacing with 3rd party electronics via (e.g.) UART, I2C, ... GPIB (this does require a high degree of expertise and cannot be automated)

These challenges are accentuated by the fact that there is little to no expertise in the company on the Accretech UF200R, electronics test and software development, and embedded systems.

A major risk with this program is disabling prober 8. It presently uses a legacy LabVIEW project running on its control computer to initiate probing. It is not presently known who, if anyone in the company can edit and optimize this script for new wafer maps and electronics readings. This was setup for probing of the 'Maddy' project and may indeed have its roots set further back in test development. The aim will be to write an application that replaces the LabVIEW project, initiates the test equipment (prober and electronics),

loads the wafer parameters and runs the test. This will control all via GPIB commands. The challenges with this will be:

1. Translating coordinate systems
 - wafer map origin vs prober origin
 - die center vs the pad origin
 - rotation direction
 - quadrant sign conventions
2. Ensuring touchdown safety
 - never move XY while in contact
 - enforce Z-safe state before motion
3. Generating a probe-card mapping system
 - pad-to-needle-to-instrument mapping must be unambiguous
4. Ensuring alignment repeatability
 - MEMS pads/marks may not match simple IC assumptions
5. UF200R command set access (We have the [manual](#))

We will need the specific UF200R GPIB remote command [manual](#) or an existing driver reference. Public pages confirm UF200R/UF-series GPIB use, but the exact command set is usually in vendor/tester documentation, Atomica does possess this, however work will need to be done to look at what drivers are necessary and if they can be sourced. By reconfiguring prober 8 to run a new initiation file and wafer maps it may be that the task is too complex and 3rd party vendors or training is necessary to get the machine working.

Mitigation

If it is the case (that a vendor needs to be contracted), Atomica management must accept that the risk is ‘taking the prober offline’ and the mitigation cost is either using 3rd party operators or the cost of training and maintaining an engineer or technician to operate the prober. In this case, it can be assumed that training in both service and operation will cost in the \$2000 to \$5000 range¹.

¹ AI review on Accretech training and costs: <https://share.google/aimode/ITPPC7OdXGinvJ7sj>

Plan

The use of AI in developing the code to run on the ‘prober master computer’ will be extensive. The process will be to initiate communications from the master computer between the prober and DAQ equipment via the GPIB (General Purpose Interface Bus) or USBTMC (USB Test and Measurement Class): the present setup will be investigated and used if deemed suitable. Programming the Accretech UF200R to handle different wafer types, developing/designing and ordering probe cards will also be documented so that a technician can perform the task following a standardized procedure. This plan/and process will be developed so that all integration engineers, TLs and layout engineers will be familiar with it. In this way probing and test can be standardized within Atomica and offered as a service during sales. The aim will be to ensure that ‘test’ is considered at the layout stage of a device so that masks are geared to the system that is developed for tests. In practical terms this means that the device layout grid, pad assignments and the (electronic) relationship between pads should be captured in a standardized format at the layout stage. At a later stage this can be developed to include a netlist for a SPICE model.

ATA (Atomica Test Application)

A wafer-test planning and automation system application will be developed as the heart of the system and will sit on the prober 8 control PC. The GUI for the application is shown in Figure 1. The application will be written to account for Atomica’s in-house test equipment (*i.e.*, electronics test equipment and the UF200R prober).

For each project, input to the software will be the wafer map, alignment marks, pad layout and the electrical model for devices. The main software behind the GUI will parse the wafer map to coordinates that can be fed directly to the UF200R. It will then generate a test plan. An instrument control layer will then take the control plan and open/setup the architecture for the computer to interface with the prober and bench top equipment. Full details on the code, its architecture and implementation of the ATA application are laid out in Appendix (i).

Vision System

Teaching technicians/operators to locate and capture the alignment marks on the wafer has proved challenging. As the UF200R has both wafer alignment through notch location (this is course) and a vision system incorporated into it for exact alignment, to reduce the burden on the operator/technician an alignment module will be incorporated into the software. In this manner wafers can be loaded into the prober and automatically probed with minimal user input.

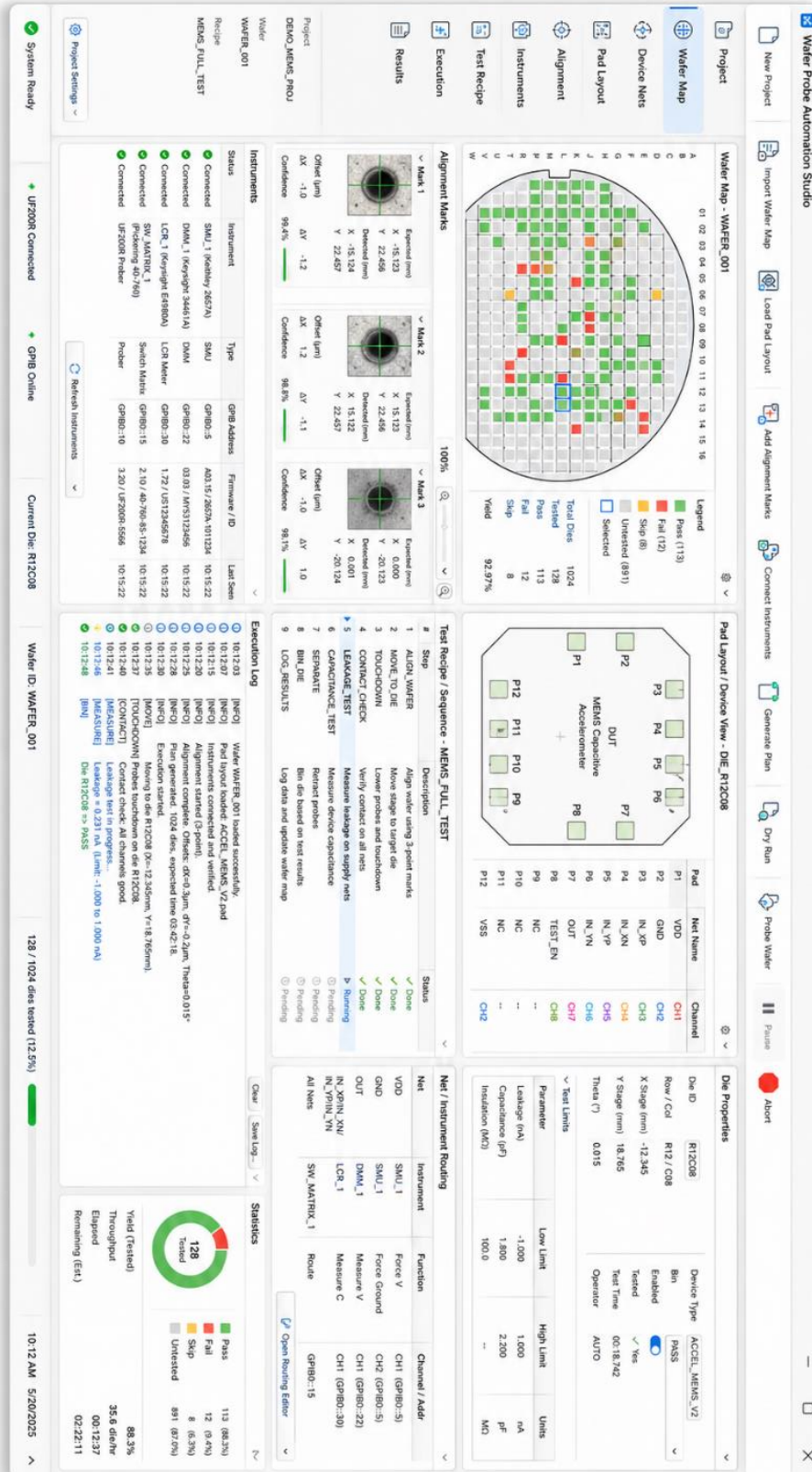


Figure 1: The proposed GUI for the ATA application

Implementation

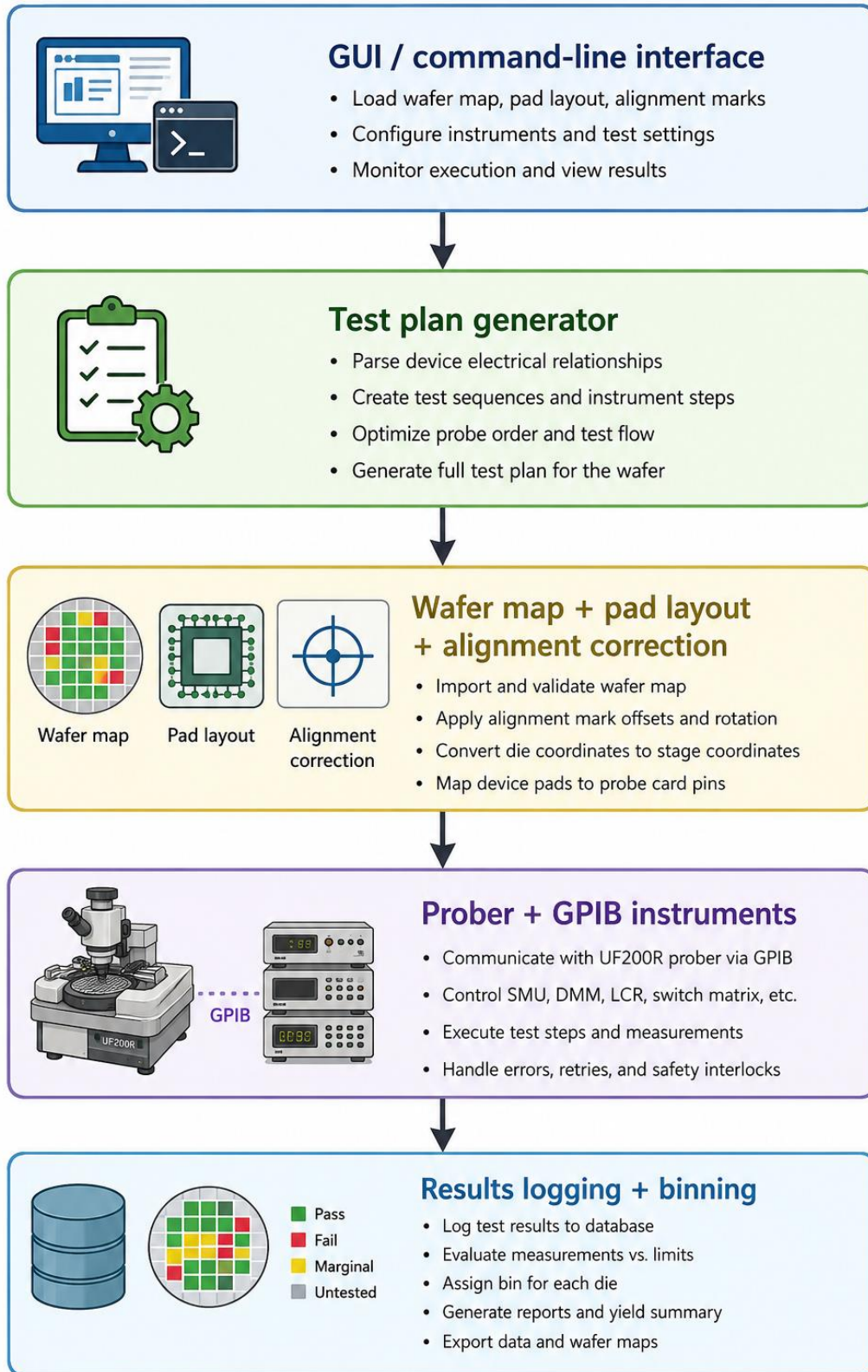


Figure 2: Software architecture in Python

The project will be implemented in the following stages

1. Establish communications between the prober computer, benchtop electronics and the UF200r to accept configuration on wafer maps and probing (Python)
2. Program the UF200r to load and align to wafer(s) and accept device coordinates
3. Perform above test procedures on dummy wafers (LCR readings)
4. Interface with USB/UART/GPIB/IEE-488 to communicate with 3rd party DAQ cards and measuring equipment and collect data (Nautilus project)

Software stack

The main software build will use Python as the 'glue' that connects everything. Python is a good fit because it has strong libraries for instrument control, file parsing, image processing, math/coordinate transforms, data logging, GUIs and databases: basically, everything we need. Figure 2 shows conceptually how each module in the ATA would work. The concept is that a single (Python) module does not directly control the prober movement, measurement logic, wafer maps and the GUI together. Instead, each part is modular.

Project structure and implementation

This project will be run in stages that build on one another. A more detailed explanation of the full development order is given in Appendix ii. Additionally, there is a full project plan for the project in the accompanying MS Word and Excel documentation².

The folder structure for the (Python-based) wafer probe automation program is as follows (remember that the main idea is that the software is split into separate blocks, so each part has a clear function):

main.py

Starts the program and connects all the pieces together.

config/

Stores setup files, e.g. which instruments are connected, the probe card wiring, device test recipes, and pad layouts.

core/

Contains the main logic of the system. It handles the wafer map, device model, pad layout, coordinate conversion, test-plan generation, test execution, and safety checks.

² "C:\Users\tniblock\OneDrive - Atomica Corporation\Desktop\Atomica\Test\WaferProbeAndTestAutomation"

instruments/

Contains the drivers for the real hardware. This includes the GPIB communication base code, SMU, DMM, LCR meter, switch matrix, and the Accrettech UF200R prober driver.

vision/

Handles camera/alignment tasks. It finds alignment marks, calibrates the camera-to-stage coordinate system, and corrects wafer position errors.

database/

Stores and exports test data. It defines the database structure, saves wafer test results, and exports reports or CSV files.

gui/

Contains the Windows-style user interface. It shows the wafer map, instrument status, execution progress, and test results.

Table 1 summarizes these blocks in simplified terms:

config/	setup files
core/	main thinking and test logic
instruments/	talks to prober and electronics
vision/	alignment mark/camera logic
database/	saves results
gui/	user interface

Table 1: A simplified explanation of each software block.

Hence, the whole program-flow from the user's perspective would be that they load a wafer or cassette into the prober and the test setup onto the prober computer. Once the user runs the ATA, it builds the test plan. With all the test data the ATA is 'ready' and upon the users command it will load a wafer, and its alignment system corrects wafer position and runs the prober and instrument test. The test results are saved and shown in the GUI. The software core would be:

- Python
- PyVISA for GPIB
- NumPy for coordinate transforms
- Pandas for wafer maps/results
- OpenCV for alignment mark recognition
- SQLite or PostgreSQL for test data

And for configuration:

- YAML or JSON for device/test definitions
- CSV or JSON for wafer maps
- INI/YAML for probe-card definitions

For the user interface we would start simple using the command line and jupyter notebook for debugging. We would then develop a small web UI using (e.g.) Flask/FastAPI. For the data output, formats would be a CSV summary file, full JSON for the result file, a SQLite database and (e.g.) wafer bin map for images. File formats that will be used are shown in Table 2:

File Format	File content
CSV	wafer map
YAML	device/test recipe
YAML	pad/probe-card map
PyVISA	GPIB drivers
OpenCV	alignment module
SQLite	results database

Table 2: File formats

Once the basic modules have been successfully implemented the next stage would then be to add:

- native Accretech map import/export
- full UI
- automatic alignment
- multi-site probing
- automatic bin-map generation
- MES/database integration

The first milestone would be to generate a wafer map and a device recipe, generating a safe, ordered probing plan and simulating every move without touching hardware. Once that works, the UF200R control and bench instrument control modules would be created and tested separately.

Finally, the complete system would be combined. As stated, the full project plan for the complete system is provided in the accompanying document (See footnote 2, page 8).

Integrating this project into existing projects (Nautilus)

The Nautilus project requires ~8k devices measured per wafer. Each device takes approximately five seconds to measure. Hence, measuring all the devices would take 40k seconds or 11.11 hours. The measurement process also requires specific device operation and tests (heating, resistance). A customized board developed by the customer (the Nanoz EK-IV) is available from to perform the test. This board can measure 2 devices in parallel. The board is connected to a PC via USB. As the probe cards can hold up to 48 pins and the devices have 5 pins, it should be possible to connect (e.g.) 9 devices at a time and reduce the probing time down to under 2 hours.

Hence, as part of this project Atomica can integrate the Nanoz EK-IV as a USB serial DAQ instrument block alongside the existing GPIB instruments. It should not be treated as a GPIB device. It should sit in the same system layer as the SMU, DMM, LCR meter, and switch matrix, but use USB/FTDI serial communication instead of PyVISA-GPIB. The EK-IV uses a simple ASCII command protocol, with commands ending in CR or LF, and the EK-IV passes through a USB interface equipped with an FTDI module. It also identifies EK-IV using whoami, which should return lam 5164.

Hence, in the system architecture would include the Nanoz in the coms and hardware control layer. The EK-IV would be a specialized measurement module connected directly to the probe card.

Appendix (i): Wafer-Test Planning and Automation System

This appendix covers software that combines wafer-map parsing, pad/probe geometry, device electrical net rules, vision/alignment data, and instrument/prober control into one test-flow generator.

1. Inputs the ATA software will need

It will be necessary to define the format of and then import the following data with each probing job:

Wafer map

- Die row/column or x/y locations
- Good/bad/skip bins
- Device type per die, if mixed
- Optional Accretech/TSK map formats, control map files, or Atomica's own CSV/JSON format
- Some UF-series integrations use map/control files, and SweepMe notes that UF-series die lists can come from a .MDF control map file while wafer lists are retrieved over GPIB. ([SweepMe!](#))

Device electrical model

Example:

```
{
  "device_type": "MEMS_resonator_A",
  "pads": {
    "P1": "drive_plus",
    "P2": "drive_minus",
    "P3": "sense_plus",
    "P4": "sense_minus",
    "P5": "substrate"
  },
  "tests": [
    {
      "name": "leakage",
      "force": "P1",
      "measure": "P2",
      "voltage": 5,
      "limit_amps": 1e-6
    },
    {
      "name": "capacitance",
      "between": ["P3", "P4"],
      "frequency": 10000
    }
  ]
}
```

}

Pad layout

- Pad coordinates relative to each die origin
- Pad names
- Probe-card channel mapping
- Probe needle or pogo pin assignments
- Multi-site offsets, if testing multiple devices/dies per touchdown

We should note that we have NI DAQ cards installed on the prober 8 computer. NI's Accretech GPIB Prober driver uses a probe-card configuration file to determine site die coordinates from current prober position, with entries like `Die01=0,0` and additional die offsets relative to the first die. This is a useful model to copy conceptually. ([NI Knowledge](#))

Alignment mark data

- Wafer-level alignment mark locations
- Die-level alignment marks, if used
- Images/templates of marks
- Camera pixel-to-stage coordinate calibration
- Rotation, scale, and translation correction

Bench equipment definition

- SMUs
- DMMs
- LCR meters
- switch matrices
- relays
- signal generators
- oscilloscopes
- any GPIB addresses and SCPI command sets

Prober definition

- Accretech UF200R GPIB address
- Chuck coordinate system
- wafer diameter
- safe Z heights
- touchdown height
- overtravel
- cleaning routine
- reject/skip behavior

2. Main software modules

A. Wafer map parser

This module will convert the wafer map into a normalized internal format:

```
Die(  
    id="R12C08",  
    row=12,  
    col=8,  
    x_mm=34.250,  
    y_mm=-12.750,  
    bin="untested",  
    device_type="MEMS_resonator_A",  
    enabled=True  
)
```

It should support:

- CSV import first
- then Accretech/TSK/UF map formats later
- manual exclusion zones
- edge die handling
- retest logic

B. Coordinate transform engine

This is critical when using the UF200R as we need to convert wafer map die coordinates to probe touch down coordinates:

- corrected wafer coordinate
- prober chuck coordinate
- probe-card touchdown coordinate

The transform should include:

```
x_stage = scale_x * (x_map cosθ - y_map sinθ) + offset_x  
y_stage = scale_y * (x_map sinθ + y_map cosθ) + offset_y
```

For Atomica MEMS wafers, we also want:

- local die alignment correction
- wafer bow compensation
- nonuniform die pitch correction
- per-quadrant alignment correction
- pad-level correction from vision

C. Vision/alignment module

This module will use Atomica mark images and known mark locations to find the real wafer position.

Workflow:

1. Move to expected alignment mark location.
2. Capture camera image.
3. Run template matching or feature detection.
4. Find actual mark center.
5. Compare actual vs expected.
6. Calculate wafer translation/rotation.
7. Optionally repeat with two or three marks.

At minimum, we will use two marks to solve translation + rotation. Three marks will let us detect scale/shear problems.

Output:

```
{  
  "offset_x_mm": 0.037,  
  "offset_y_mm": -0.021,  
  "rotation_deg": 0.018,  
  "confidence": 0.96  
}
```

D. Test-plan generator

This is the brain or logic that performs the test, i.e. do this then that...It takes:

- wafer map
- device electrical model
- pad layout
- probe card layout
- test equipment available (can be Coms to a dedicated circuit with (e.g.) a DAC)
- binning rules

and generates a sequence like:

1. Load wafer
2. Align wafer
3. Move to die R01C01
4. Touch down
5. Connect pad P1 to SMU1
6. Connect pad P2 to SMU2
7. Run leakage test
8. Run capacitance test
9. Save result
10. Bin die
11. Lift probes
12. Move to next die

The generated test regime should probably be stored as something readable like YAML:

```
wafer_id: WAFER_001
test_plan:
  - action: align_wafer
    marks: [M1, M2, M3]

  - action: move_to_die
    die: R12C08

  - action: touchdown
    z_mode: contact

  - action: configure_switch_matrix
    connections:
      SMU1_HI: P1
      SMU1_LO: P2

  - action: measure_iv
    sweep:
      start_v: -5
      stop_v: 5
      step_v: 0.1
```

Atomica Corporation

```
    compliance_a: 0.001  
- action: bin_die  
  rules:  
    pass_if:  
      leakage_a_max: 1e-6
```

E. Instrument-control layer

This module will use Python + PyVISA for GPIB/USB/LAN instruments.

The structure will be:

```
import pyvisa

rm = pyvisa.ResourceManager()

dmm = rm.open_resource("GPIB0::22::INSTR")
smu = rm.open_resource("GPIB0::24::INSTR")
prober = rm.open_resource("GPIB0::5::INSTR")
```

The Accrettech/TSK prober needs to be configured properly for GPIB operation. A Keithley S530 prober-driver manual notes that the prober GPIB address is commonly set to 5 unless another address is used, and that setup must be done from the prober configuration menus after boot. ([Tequipment](#)). The present configuration will be investigated and updated as needed.

F. Prober driver abstraction

We will not scatter raw GPIB prober commands throughout our code. Instead, we shall make a clean prober driver class:

```
class Prober:
    def load_wafer(self):
        pass

    def unload_wafer(self):
        pass

    def move_to_die(self, die):
        pass

    def move_relative(self, dx_mm, dy_mm):
        pass

    def touchdown(self):
        pass

    def separate(self):
        pass

    def clean_needles(self):
        pass

    def get_position(self):
        pass

    def get_wafer_id(self):
        pass
```

Then make one implementation:

```
class AccretechUF200R(Prober):
    def __init__(self, visa_resource):
        self.inst = visa_resource

    def move_to_die(self, die):
        # send UF200R-specific GPIB command here
        pass
```

That way, later we can support another prober without rewriting the test logic.

3. Important safety/state logic

We absolutely want a state machine for the prober and test system.

Example states:

IDLE
LOADED
ALIGNED
READY_TO_PROBE
MOVING
CONTACT
TESTING
SEPARATED
ERROR
ABORTED

Before any test action, check:

Is wafer loaded?
Is wafer aligned?
Is chuck at safe Z?
Are probes separated?
Is target die valid?
Are instruments initialized?
Are compliance limits set?

This prevents expensive mistakes like moving laterally while probes are still in contact.

Appendix ii

Practical development order

The plan will be not to attempt full automation from the beginning of the project but have a phased approach as laid out in the following phases. Within this plan the Nautilus project will be integrated to leverage the use of the prober and the test circuit that the customer has already developed:

Phase 1: Offline test-plan generator

No hardware control yet.

Input:

wafer map + pad layout + device test rules

Output:

ordered die list + planned measurements + expected prober moves

This lets you debug the logic safely.

Phase 2: Instrument-only control

Control your DMM/SMU/LCR/switch matrix over GPIB without moving the prober.

Test:

- voltage sweep
- current measurement
- capacitance measurement
- relay switching
- error handling
- compliance limits

Phase 3: Prober dry-run mode

Connect to the UF200R, but do not touch down.

Commands:

```
load wafer
read wafer ID
move to die
move to next die
separate
unload
```

Log every move.

Phase 4: Single-die probing

Manually align one die, then let software:

```
touch down → measure → separate
```

Do not scan the whole wafer yet.

Phase 5: Alignment correction

Add vision alignment:

```
find marks → calculate wafer transform → verify target die
```

Phase 6: Full-wafer automated probing

Only after all the above works.

The key data model

The most important thing is to separate **device pads** from **instrument channels**.

Example:

```
Device pad name:  DRIVE+
Physical pad:     pad 3 on die
Probe needle:     probe_card_site_1_pin_7
Switch matrix:    row 7
Instrument:        SMU1 HI
```


Atomica Corporation

So the chain is:

```
Electrical function
→ die pad
→ physical pad coordinate
→ probe needle/channel
→ switch matrix path
→ instrument terminal
```

That lets the software generate tests automatically.

Example generated test flow

For one die:

```
Move to die R14C22
Verify die is enabled in wafer map
Apply alignment correction
Move chuck to corrected x/y
Lower to contact
Wait 200 ms
Measure contact check
If contact fail:
    retry touchdown once
    if still fail, bin as CONTACT_FAIL
Configure switch matrix for leakage test
Set SMU compliance
Sweep voltage
Record current
Configure LCR meter
Measure capacitance
Lift probes
Bin die
Move to next die
```

Best overall design

The cleanest design is:

```
WaferMap
DeviceDefinition
PadLayout
ProbeCard
AlignmentModel
TestRecipe
InstrumentDrivers
ProberDriver
TestExecutor
```

Atomica Corporation

ResultDatabase

Where the final executor does:

```
for die in wafer_map.enabled_dies():
    prober.move_to_die(die)
    prober.touchdown()

    for test in recipe.tests_for(die.device_type):
        instruments.configure(test)
        result = instruments.run(test)
        database.save(die, test, result)

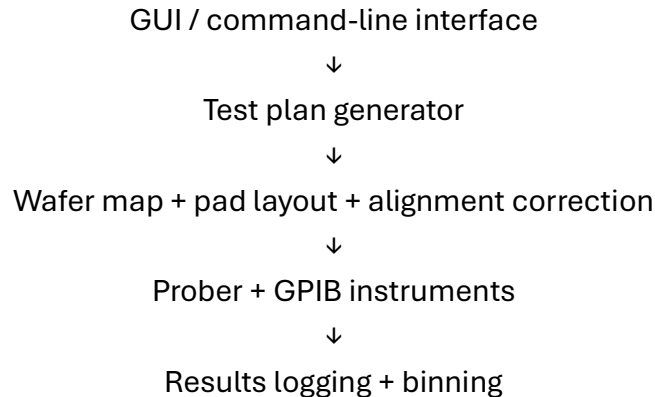
prober.separate()
```

Appendix iii Full code simulation

Below is a more detailed breakdown of the Core software blocks for the ATA wafer-probing automation system.

1. Python: the main application/control language

Python would be the “glue” that connects everything:



Python is a good fit because it has strong libraries for:

- instrument control
- file parsing
- image processing
- math/coordinate transforms
- data logging
- GUIs
- databases

A good project structure might look like this:

```
wafer_probe_automation/  
|  
├─ main.py  
├─ config/  
|   ├── instruments.yaml  
|   ├── probe_card.yaml  
|   ├── device_recipes/  
|   └─ pad_layouts/  
└─ core/
```

```
|  ├── wafer_map.py
|  ├── device_model.py
|  ├── pad_layout.py
|  ├── coordinate_transform.py
|  ├── test_plan.py
|  ├── executor.py
|  └── safety.py
|
|  ├── instruments/
|  |   ├── gpib_base.py
|  |   ├── smu.py
|  |   ├── dmm.py
|  |   ├── lcr_meter.py
|  |   ├── switch_matrix.py
|  |   └── accrettech_uf200r.py
|  |
|  |   ├── vision/
|  |   |   ├── alignment.py
|  |   |   ├── mark_detection.py
|  |   |   └── calibration.py
|  |   |
|  |   ├── database/
|  |   |   ├── schema.sql
|  |   |   ├── results_db.py
|  |   |   └── export.py
|  |   |
|  |   └── gui/
|  |       ├── app.py
|  |       ├── wafer_map_view.py
|  |       ├── instrument_panel.py
|  |       └── execution_panel.py
```

The most important design idea is this:

Python should not directly mix proper movement, measurement logic, wafer maps, and GUI code together.

Instead, each part should be modular.

For example:

```
die = wafer_map.get_die("R12C08")
target_position = transform.die_to_stage(die)

prober.move_to(target_position.x, target_position.y)
prober.touchdown()

result = test_executor.run_tests_for_die(die)

prober.separate()
database.save_result(result)
```

That kind of structure keeps the system readable and safe.

2. PyVISA for GPIB control

PyVISA is the layer that lets Python talk to instruments over:

- GPIB
- USB-TMC
- Serial
- Ethernet/LAN
- VXI-11
- HiSLIP

For your system, it would mainly talk to:

- Accrettech UF200R prober
- SMU
- DMM
- LCR meter
- switch matrix
- relay controller
- signal generator
- oscilloscope

A typical PyVISA setup looks like:

```
import pyvisa

rm = pyvisa.ResourceManager()
```

```
smu = rm.open_resource("GPIB0::5::INSTR")
dmm = rm.open_resource("GPIB0::22::INSTR")
lcr = rm.open_resource("GPIB0::30::INSTR")
prober = rm.open_resource("GPIB0::10::INSTR")
```

Then you send commands:

```
smu.write("*RST")
smu.write(":SOUR:VOLT 5")
smu.write(":OUTP ON")
current = smu.query(":MEAS:Curr?")
```

But in the actual system, you should **not** write raw commands everywhere. Instead, wrap each instrument in a class.

Example:

```
class SMU:
    def __init__(self, visa_resource):
        self.inst = visa_resource

    def reset(self):
        self.inst.write("*RST")

    def force_voltage(self, voltage, compliance_current):
        self.inst.write(f":SOUR:VOLT {voltage}")
        self.inst.write(f":SENS:Curr:PROT {compliance_current}")
        self.inst.write(":OUTP ON")

    def measure_current(self):
        return float(self.inst.query(":MEAS:Curr?"))

    def output_off(self):
        self.inst.write(":OUTP OFF")
```

Then the test code becomes much cleaner:

```
smu.force_voltage(5.0, compliance_current=1e-6)
leakage = smu.measure_current()
smu.output_off()
```

For the prober, the same idea applies:

Atomica Corporation

```
class AccretechUF200R:
    def __init__(self, visa_resource):
        self.inst = visa_resource

    def move_to_die(self, row, col):
        # UF200R-specific GPIB command goes here
        pass

    def touchdown(self):
        pass

    def separate(self):
        pass

    def get_position(self):
        pass
```

The exact UF200R commands would depend on the Accretech remote/GPIB command manual.

What PyVISA block is responsible for

- Connecting to instruments
- Checking instrument identity
- Sending SCPI/GPIB commands
- Reading measurements
- Handling timeouts
- Handling communication errors
- Resetting instruments
- Logging raw commands if needed

Example instrument configuration file:

```
instruments:
  prober:
    type: AccretechUF200R
    address: GPIB0::10::INSTR

  smu_1:
    type: KeithleySMU
    address: GPIB0::5::INSTR
```

```
dmm_1:
  type: KeysightDMM
  address: GPIB0::22::INSTR

lcr_1:
  type: KeysightLCR
  address: GPIB0::30::INSTR

switch_matrix:
  type: PickeringSwitchMatrix
  address: GPIB0::15::INSTR
```

The software would load this file and automatically connect to everything.

3. NumPy for coordinate transforms

NumPy is mainly for the math.

In the system, it would handle conversion between:

- wafer map coordinates
- die row/column coordinates
- physical wafer x/y
- alignment-corrected x/y
- prober stage x/y
- pad coordinates
- probe-card coordinates

This is one of the most important parts of the whole project.

Example coordinate problem

The wafer map may say:

```
Die R12C08 is at:
x = -12.300 mm
y = 18.700 mm
```

But after vision alignment, the wafer is found to be slightly shifted and rotated:

```
x offset = +0.035 mm
y offset = -0.021 mm
rotation = 0.018 degrees
```


Hence, the prober should not move to the raw wafer-map location. It should move to the corrected location.

A simple transform might be:

```
import numpy as np

def transform_point(x, y, dx, dy, theta_deg):
    theta = np.deg2rad(theta_deg)

    rotation = np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta),  np.cos(theta)]
    ])

    point = np.array([x, y])
    offset = np.array([dx, dy])

    corrected = rotation @ point + offset
    return corrected[0], corrected[1]
```

Usage:

```
x_stage, y_stage = transform_point(
    x=-12.300,
    y=18.700,
    dx=0.035,
    dy=-0.021,
    theta_deg=0.018
)
```

For a real wafer, you may eventually need more than a simple rotation/translation model.

Possible correction models:

2-point alignment:
translation + rotation

3-point alignment:
translation + rotation + scale/shear check

Multi-point alignment:
local distortion correction

Atomica Corporation

Per-die vision:

highest accuracy but slower

Coordinate transform block should own

- Die pitch
- wafer origin
- notch/flat orientation
- map row/column convention
- stage coordinate convention
- alignment offsets
- rotation correction
- pad offsets inside each die
- multi-site probe-card offsets
- safe movement locations

Example class:

```
class CoordinateTransform:
    def __init__(self, die_pitch_x, die_pitch_y):
        self.die_pitch_x = die_pitch_x
        self.die_pitch_y = die_pitch_y
        self.offset_x = 0.0
        self.offset_y = 0.0
        self.theta_deg = 0.0

    def set_alignment_correction(self, offset_x, offset_y,
theta_deg):
        self.offset_x = offset_x
        self.offset_y = offset_y
        self.theta_deg = theta_deg

    def row_col_to_wafer_xy(self, row, col, origin_row,
origin_col):
        x = (col - origin_col) * self.die_pitch_x
        y = (row - origin_row) * self.die_pitch_y
        return x, y

    def wafer_xy_to_stage_xy(self, x, y):
        return transform_point(
            x,
            y,
```

```
        self.offset_x,  
        self.offset_y,  
        self.theta_deg  
    )
```

Then:

```
x, y = transform.row_col_to_wafer_xy(row=12, col=8,  
origin_row=10, origin_col=10)  
x_stage, y_stage = transform.wafer_xy_to_stage_xy(x, y)
```

This block is also where you would prevent dangerous moves, for example:

- Do not move outside wafer boundary.
- Do not move to disabled die.
- Do not move while probes are down.
- Do not move beyond known chuck limits.

4. Pandas for wafer maps and results

Pandas is useful because wafer maps and test results are naturally table-like.

A wafer map could be stored as a DataFrame:

```
import pandas as pd
```

```
wafer_map = pd.DataFrame([  
    {"die_id": "R01C01", "row": 1, "col": 1, "x_mm": -30.0,  
    "y_mm": 25.0, "status": "untested"},  
    {"die_id": "R01C02", "row": 1, "col": 2, "x_mm": -27.5,  
    "y_mm": 25.0, "status": "skip"},  
    {"die_id": "R12C08", "row": 12, "col": 8, "x_mm": -12.3,  
    "y_mm": 18.7, "status": "untested"},  
])
```

Then you can filter easily:

```
enabled_dies = wafer_map[wafer_map["status"] != "skip"]
```

Sort the probing order:

```
probe_order = enabled_dies.sort_values(["row", "col"])
```

Or use serpentine movement to reduce travel:

Atomica Corporation

```
def serpentine_sort(df):
    rows = []
    for row_num, group in df.groupby("row"):
        if row_num % 2 == 0:
            rows.append(group.sort_values("col"))
        else:
            rows.append(group.sort_values("col",
ascending=False))
    return pd.concat(rows)
```

That would generate a path like:

- Row 1: left to right
- Row 2: right to left
- Row 3: left to right

which is often faster than returning to the same side every row.

Results table

Each test result can also be stored in Pandas:

```
results = pd.DataFrame([
    {
        "wafer_id": "WAFER_001",
        "die_id": "R12C08",
        "test_name": "leakage",
        "value": 0.231,
        "units": "nA",
        "low_limit": -1.0,
        "high_limit": 1.0,
        "result": "PASS"
    }
])
```

Then you can calculate:

```
pass_count = len(results[results["result"] == "PASS"])
fail_count = len(results[results["result"] == "FAIL"])
yield_percent = pass_count / (pass_count + fail_count) * 100
```

Pandas is also useful for exporting:

```
results.to_csv("wafer_001_results.csv", index=False)
wafer_map.to_csv("wafer_001_final_bins.csv", index=False)
```

Pandas block should handle

- Import wafer maps
- Export wafer maps
- Store die list
- Store per-die status
- Merge test results with die coordinates
- Generate summary statistics
- Generate bin maps
- Support retesting and filtering

For example:

```
class WaferMap:
    def __init__(self, csv_path):
        self.df = pd.read_csv(csv_path)

    def enabled_dies(self):
        return self.df[self.df["enabled"] == True]

    def untested_dies(self):
        return self.df[self.df["bin"] == "UNTESTED"]

    def update_die_bin(self, die_id, new_bin):
        self.df.loc[self.df["die_id"] == die_id, "bin"] =
new_bin

    def save(self, path):
        self.df.to_csv(path, index=False)
```

5. OpenCV for alignment mark recognition

OpenCV would be used for the visual alignment part.

The goal is:

- Given a camera image from the prober,

- find the actual alignment mark position,
- compare it to the expected position,
- then correct the wafer coordinate system.

Basic flow:

- Move probe to expected alignment mark location
- Capture image
- Find mark in image
- Calculate pixel offset
- Convert pixel offset to microns/mm
- Repeat for another mark
- Calculate wafer shift and rotation
- Apply correction to all future moves

Simple template matching

If your alignment mark shape is consistent, you can use template matching.

```
import cv2
import numpy as np

image = cv2.imread("camera_image.png", cv2.IMREAD_GRAYSCALE)
template = cv2.imread("alignment_mark_template.png",
cv2.IMREAD_GRAYSCALE)

result = cv2.matchTemplate(image, template,
cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(result)

print("Best match confidence:", max_val)
print("Top-left pixel:", max_loc)
```

Then get center:

```
template_h, template_w = template.shape
center_x = max_loc[0] + template_w / 2
center_y = max_loc[1] + template_h / 2
```

If the expected mark center was image center:

```
image_h, image_w = image.shape
expected_x = image_w / 2
```

Atomica Corporation

```
expected_y = image_h / 2

dx_pixels = center_x - expected_x
dy_pixels = center_y - expected_y

Convert to physical offset:

microns_per_pixel = 1.25

dx_um = dx_pixels * microns_per_pixel
dy_um = dy_pixels * microns_per_pixel
```

Then:

The mark is +4.2 μm in X and -1.7 μm in Y from expected.

More robust mark detection

Template matching is the simplest. But for production, you may want:

- Thresholding
- edge detection
- circle/line detection
- contour detection
- fiducial cross detection
- feature matching
- subpixel centroid fitting

For cross-shaped alignment marks, you might:

1. threshold the image
2. find contours
3. identify the cross shape
4. compute centroid
5. refine with subpixel fitting

Example conceptual pipeline:

```
gray = cv2.imread("mark.png", cv2.IMREAD_GRAYSCALE)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)

contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

Choose contour with correct area/shape

OpenCV block should own

Camera image acquisition interface

Template image storage

Mark detection algorithm

Confidence score

Pixel-to-micron calibration

Expected vs detected mark position

Wafer offset calculation

Rotation calculation

Image logging for traceability

Example output from alignment module:

```
{  
  "mark_id": "M1",  
  "expected_x_mm": -15.123,  
  "expected_y_mm": 22.456,  
  "detected_x_mm": -15.124,  
  "detected_y_mm": 22.457,  
  "dx_um": -1.0,  
  "dy_um": 1.0,  
  "confidence": 0.994  
}
```

Then the transform module uses this to correct stage movement.

6. SQLite or PostgreSQL for test data

You need a database because CSV files alone become messy once you have:

many wafers

many devices per wafer

many tests per die

many instruments

retests

failed tests

alignment logs

operator notes
raw measurement sweeps
binning history

SQLite

SQLite is best for a first prototype.

Advantages:

- single file database
- easy to back up
- no server required
- works well for local lab software
- easy to inspect

Example:

wafer_test_results.db

Good for:

one computer
one prober
local development
prototype system

PostgreSQL

PostgreSQL is better later if you need:

multiple probers
multiple users
networked test database
web dashboard
long-term manufacturing data
role-based access
high-volume data

For your first version, I would use **SQLite**, then design the code so it can move to PostgreSQL later.

Suggested database tables

wafers

```
CREATE TABLE wafers (  
  wafer_id TEXT PRIMARY KEY,  
  lot_id TEXT,  
  device_type TEXT,  
  diameter_mm REAL,  
  created_at TEXT,  
  operator TEXT  
);
```

dies

```
CREATE TABLE dies (  
  die_id TEXT,  
  wafer_id TEXT,  
  row INTEGER,  
  col INTEGER,  
  x_mm REAL,  
  y_mm REAL,  
  bin TEXT,  
  enabled INTEGER,  
  PRIMARY KEY (wafer_id, die_id)  
);
```

tests

```
CREATE TABLE tests (  
  test_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  wafer_id TEXT,  
  die_id TEXT,  
  test_name TEXT,  
  start_time TEXT,  
  end_time TEXT,  
  result TEXT  
);
```

measurements

```
CREATE TABLE measurements (  
  measurement_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  test_id INTEGER,
```

```
parameter TEXT,  
value REAL,  
units TEXT,  
low_limit REAL,  
high_limit REAL,  
pass_fail TEXT  
);
```

alignment_runs

```
CREATE TABLE alignment_runs (  
  alignment_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  wafer_id TEXT,  
  timestamp TEXT,  
  offset_x_um REAL,  
  offset_y_um REAL,  
  theta_deg REAL,  
  confidence REAL  
);
```

instrument_log

```
CREATE TABLE instrument_log (  
  log_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  timestamp TEXT,  
  instrument_name TEXT,  
  address TEXT,  
  command TEXT,  
  response TEXT,  
  status TEXT  
);
```

This lets you answer useful questions later:

Which dies failed leakage?

Which wafers had bad alignment?

Which instrument was used?

What was the exact test recipe?

Was this a first test or a retest?

What was the yield by wafer quadrant?

Example Python database insert

```
import sqlite3
from datetime import datetime

conn = sqlite3.connect("wafer_test_results.db")
cur = conn.cursor()

cur.execute("""
INSERT INTO measurements
(test_id, parameter, value, units, low_limit, high_limit, pass_fail)
VALUES (?, ?, ?, ?, ?, ?, ?)
""", (
    101,
    "leakage",
    0.231,
    "nA",
    -1.0,
    1.0,
    "PASS"
))

conn.commit()
conn.close()
```

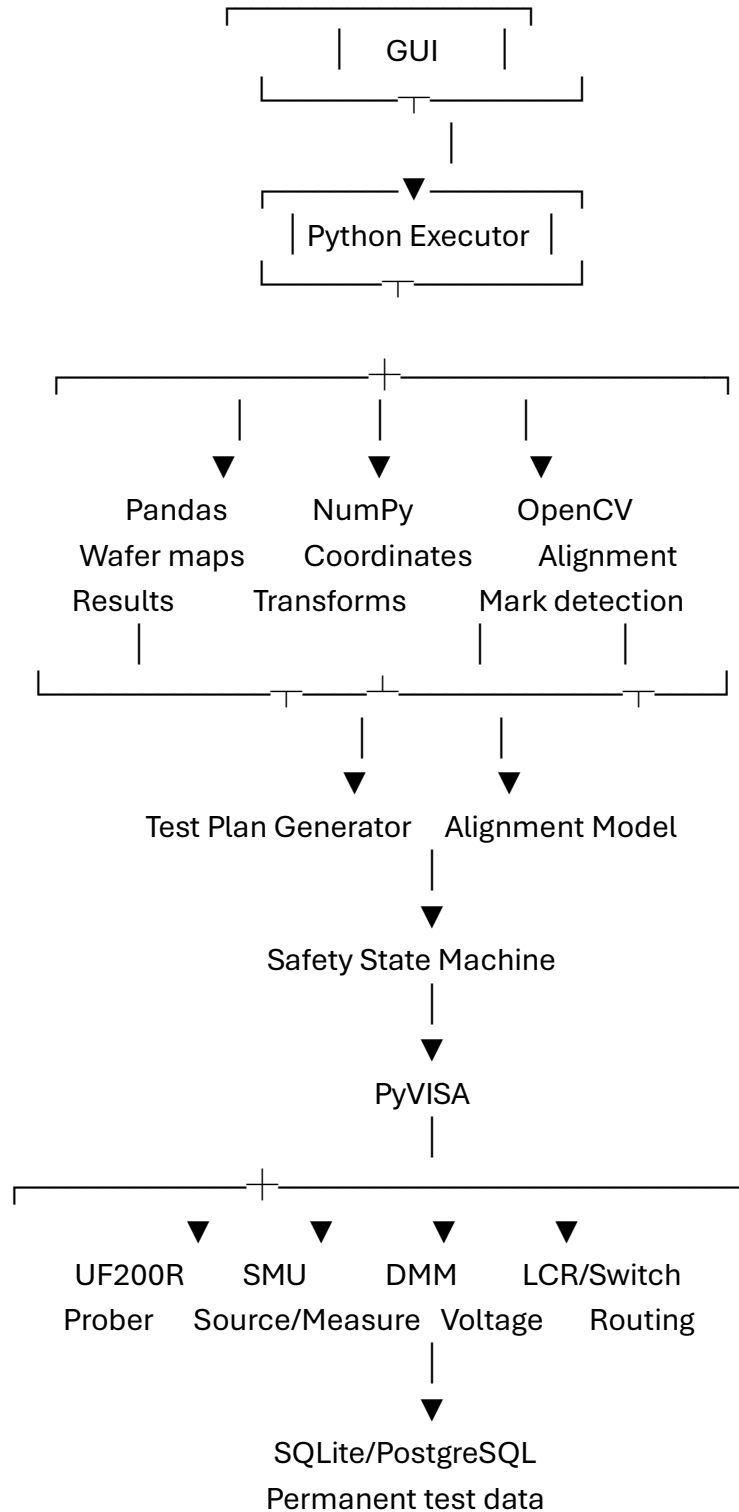
How the blocks work together

A full test flow could look like this:

1. Python application starts.
2. PyVISA connects to UF200R, SMU, DMM, LCR meter, and switch matrix.
3. Pandas imports wafer map.
4. Pandas imports pad/test data tables.
5. OpenCV finds alignment marks.
6. NumPy calculates wafer translation and rotation correction.
7. Python test planner creates die-by-die test sequence.
8. PyVISA commands prober to move to first die.
9. PyVISA commands instruments to run measurement.
10. Pandas stores live result table.

11. SQLite stores permanent result.
12. GUI updates wafer map and execution log.

Visually:



What should be built first?

For version 1, the project should be kept simple:

- Python
- PyVISA
- NumPy
- Pandas
- SQLite

This means that alignment would be semi-manual at first (i.e. no OpenCV).

Version 1 should do this:

- Load wafer map
- Load pad layout
- Load test recipe
- Connect to instruments
- Generate test plan
- Run dry simulation
- Move prober without touchdown
- Run one test on one die
- Save results

Then version 2 adds:

- Automatic alignment mark detection with OpenCV
- Full-wafer probing
- Automatic binning
- Live wafer-map GUI
- Retest flow

The most important first milestone is not the GUI. It is a safe dry-run executor that can generate every prober movement and instrument command without physically touching the wafer